Module 5 Activity 5: Working with Global Functions

Purpose:

Incorporate the ***arcpy.sa*** module to work with global functions to generate a suitability analysis

Instructions:

Any Python environment will work for this exercise.

Download the data ***Module 5 Activity 5*** and save the data to your working directory.

1. Import the ***arcpy*** module
2. Import the ***arcpy.sa*** module
3. Check out the spatial analyst extension so it can be used
4. Add a ***print*** statement to the ***CheckOutExtension*** function to notify the user that it is ready for use
5. Set the workspace and overwrite outputs
6. Create a list of raster datasets present in the geodatabase
7. Describe the raster properties of each raster
    - Format
    - Cell size
    - Spatial reference
8. Create a list of feature classes present in the geodatabase

Python

```
#import arcpy and spatial analyst modules
import arcpy
from arcpy import env
from arcpy.sa import *
#check out the spatial analyst extension
arcpy.CheckOutExtension("Spatial")
print ("Spatial Analyst extension checked out and ready to use.")
print ("")
#set the overwrite outputs and workspace
env.overwriteOutput = True
env.workspace = "C:\\GEOS456\\GlobalStats.gdb"
#get a list of the rasters in the geodatabase
rasterlist = arcpy.ListRasters()
#print statement to show what rasters are inside the geodatabase
print ("The following rasters are present in the gdb:")
#start a for loop to iterate through the list
for rasters in rasterlist:
    desc = arcpy.Describe(rasters)
    print ("-",rasters, "|","Coordinate System:", desc.spatialReference.name,\
    "|","Cell Size:",desc.meanCellWidth)
print ("")
#get a list of feature classes in the geodatabase
featurelist = arcpy.ListFeatureClasses()
#print statement to show what features are inside the geodatabase
print ("The following feature classes are present in the gdb:")
#start a for loop to iterate through the list
for features in featurelist:
    desc1 = arcpy.Describe(features)
    print ("-",features, "|",desc1.shapeType, "|",\
    "Coordinate System:", desc1.spatialReference.name)
```

The next steps will use the ***Distance*** toolset to generate a new distance rasters with the ***Euclidean Distance*** tool. We will run the ***Euclidean Distance*** tool 3 times.

9. Create an environment object for the processing extent
10. Define a variable for each required parameter of the ***Euclidean Distance*** tool
11. Apply the ***Euclidean Distance*** tool and specify a cell size of ***25***
12. Save all the rasters
13. Use a ***print*** statement to tell the user that the table has been created

Python

```
#import arcpy and spatial analyst modules
import arcpy
from arcpy import env
from arcpy.sa import *
#check out the spatial analyst extension
arcpy.CheckOutExtension("Spatial")
print ("Spatial Analyst extension checked out and ready to use.")
print ("")
#set the overwrite outputs and workspace
env.overwriteOutput = True
env.workspace = "C:\\GEOS456\\GlobalStats.gdb"
#get a list of the rasters in the geodatabase
rasterlist = arcpy.ListRasters()
#print statement to show what rasters are inside the geodatabase
print ("The following rasters are present in the gdb:")
#start a for loop to iterate through the list
for rasters in rasterlist:
    desc = arcpy.Describe(rasters)
    print ("-",rasters, "|","Coordinate System:", desc.spatialReference.name,\
    "|","Cell Size:",desc.meanCellWidth)
print ("")
#get a list of feature classes in the geodatabase
featurelist = arcpy.ListFeatureClasses()
#print statement to show what features are inside the geodatabase
print ("The following feature classes are present in the gdb:")
#start a for loop to iterate through the list
for features in featurelist:
    desc1 = arcpy.Describe(features)
    print ("-",features, "|",desc1.shapeType, "|",\
    "Coordinate System:", desc1.spatialReference.name)
#set the all the required processing enviornments
env.extent = "elevation"
#define the parameters for each of the features using the euclidean distance tool
InDataUrban = "urban"
OutEucUrban = "DistUrban"
InDataHigh = "highways"
OutEucHigh = "DistHigh"
InDataParks = "parks"
OutEucParks ="DistPark"
#apply the euclidean distance tool
OutDistRas = EucDistance(InDataUrban,"",25)
OutDistRas1 = EucDistance(InDataHigh,"",25)
OutDistRas2 = EucDistance(InDataParks,"",25)
#save the rasters
OutDistRas.save(OutEucUrban)
OutDistRas1.save(OutEucHigh)
OutDistRas2.save(OutEucParks)
print ("")
#print a message that the tool was sucessful
print ("Euclidean distances created and the following rasters were created:")
print (OutDistRas, "|","Cell Size:",desc.meanCellWidth)
print (OutDistRas1, "|","Cell Size:",desc.meanCellWidth)
print (OutDistRas2, "|","Cell Size:",desc.meanCellWidth)
print ("")
```

The next steps will use the ***Overlay*** toolset to generate a new distance rasters with the ***Fuzzy Membership*** tool. We will run the ***Fuzzy Membership*** tool 4 times.

14. Save all the resulting raster datasets
15. Use a ***print*** statement to show the results
16. Apply the ***Fuzzy Overlay*** tool to combine all the rasters into one
17. Use a ***print*** statement to show the results
18. Check the spatial analyst extension back in to the license manager

19. Run the script and examine you results

Python

```
#print a message that the tool was sucessful
print ("Euclidean distances created and the following rasters were created:")
print (OutDistRas, "|","Cell Size:",desc.meanCellWidth)
print (OutDistRas1, "|","Cell Size:",desc.meanCellWidth)
print (OutDistRas2, "|","Cell Size:",desc.meanCellWidth)
print ("")
#use the fuzzy membership tool to determine a low membership to urban areas and
#highways
FuzzyLow = FuzzyMembership(OutDistRas, FuzzySmall)
FuzzyLow1 = FuzzyMembership(OutDistRas1, FuzzySmall)
#use the fuzzy membership tool to determine a high membership to parks
FuzzyHigh = FuzzyMembership(OutDistRas2, FuzzyLarge)
#use the fuzzy membership tool to determine a near membership to elevation
FuzzyElev = FuzzyMembership(inRaster, FuzzyNear)
#save all the raster outputs
FuzzyLow.save("UrbanLow")
FuzzyLow1.save("HighwayLow")
FuzzyHigh.save("ParksHigh")
FuzzyElev.save("ElevNear")
#print a message that the tool was sucessful
print ("The following fuzzy rasters where created:")
print (FuzzyLow)
print (FuzzyLow1)
print (FuzzyHigh)
print (FuzzyElev)
print ("")
#overlay all the outputs into one raster showing the most suitable
#locations for new development using the "AND" overlay type
OutFuzzyOverlay = FuzzyOverlay([FuzzyLow,FuzzyLow1, \
FuzzyHigh, FuzzyElev], "AND")
#save the overlay raster
OutFuzzyOverlay.save("FinalOverlay")
#print a message that the tool was sucessful
print ("The final fuzzy raster was created:")
print (OutFuzzyOverlay)
print ("")
#check out the spatial analyst extension
arcpy.CheckInExtension("Spatial")
print ("Spatial Analyst returned to license manager.")
```

In previous courses, you learned the weighted overlay tools. This activity takes the same approach, but uses the Fuzzy Overlay instead. For more information on the fuzzy overlay tools, read the following links:

How Fuzzy Membership works

How Fuzzy Overlay works